

SCHEMAP DOCUMENTATION

User Guide

Clean Technical edition for the current macOS app implementation.

Build Tab	Variants	Exports	Registry
------------------	-----------------	----------------	-----------------

Generated: February 24, 2026

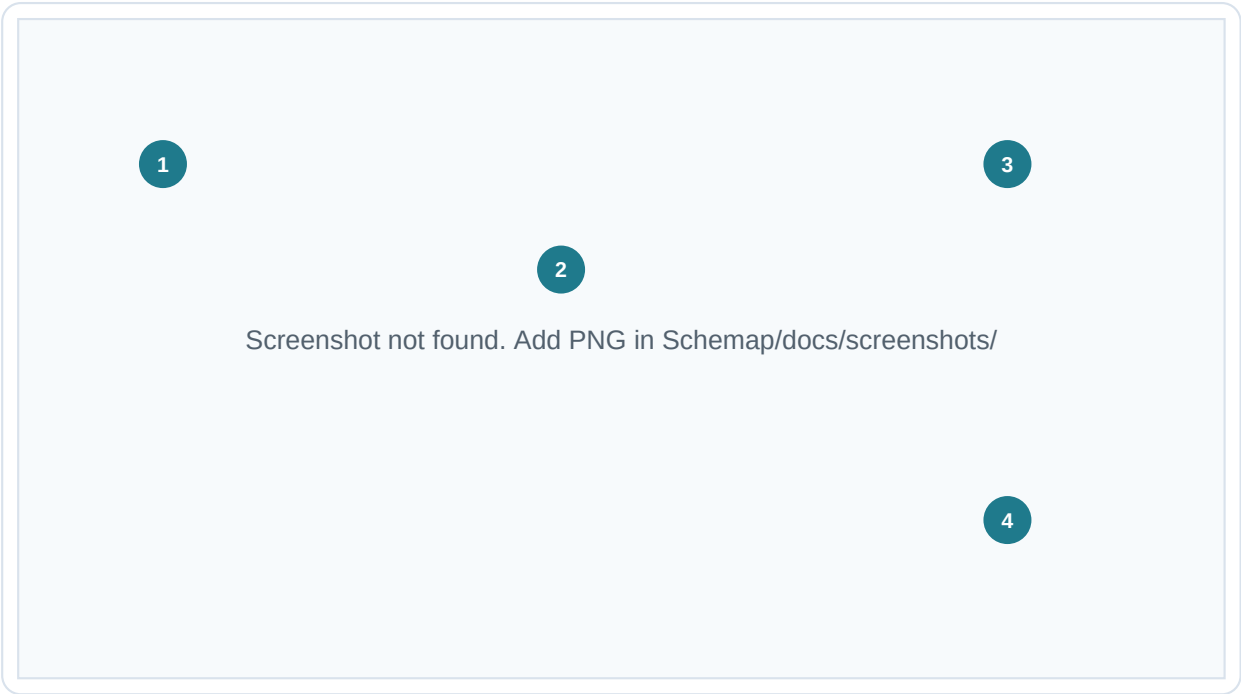
Guide Map

Use this map to jump directly to the workflow you need.

- 1** Visual Walkthrough Placeholders
- 2** Application Overview
- 3** Interface Overview
- 4** Getting Started Tutorial
- 5** Block Library Usage
- 6** Canvas Composition Workflow
- 7** Preview Stage
- 8** Asset/Object Registry
- 9** Exporting Prompts
- 10** Best Practices
- 11** Keyboard Shortcuts
- 12** Advanced Features
- 13** Troubleshooting
- 14** Next Steps

Build Tab Overview

Three-pane workspace for block composition, preview, and registry.



1	Block library: Browse categories and add blocks.
2	Composition canvas: Arrange and edit blocks in sequence.
3	Preview: Review compiled prompt output and token estimate.
4	Registry: Manage reusable projects, snippets, and policies.

Expected screenshot file(s): 01-build-overview.png

Editing Blocks In Canvas

Inline placeholders and drag-based ordering in the Build workflow.

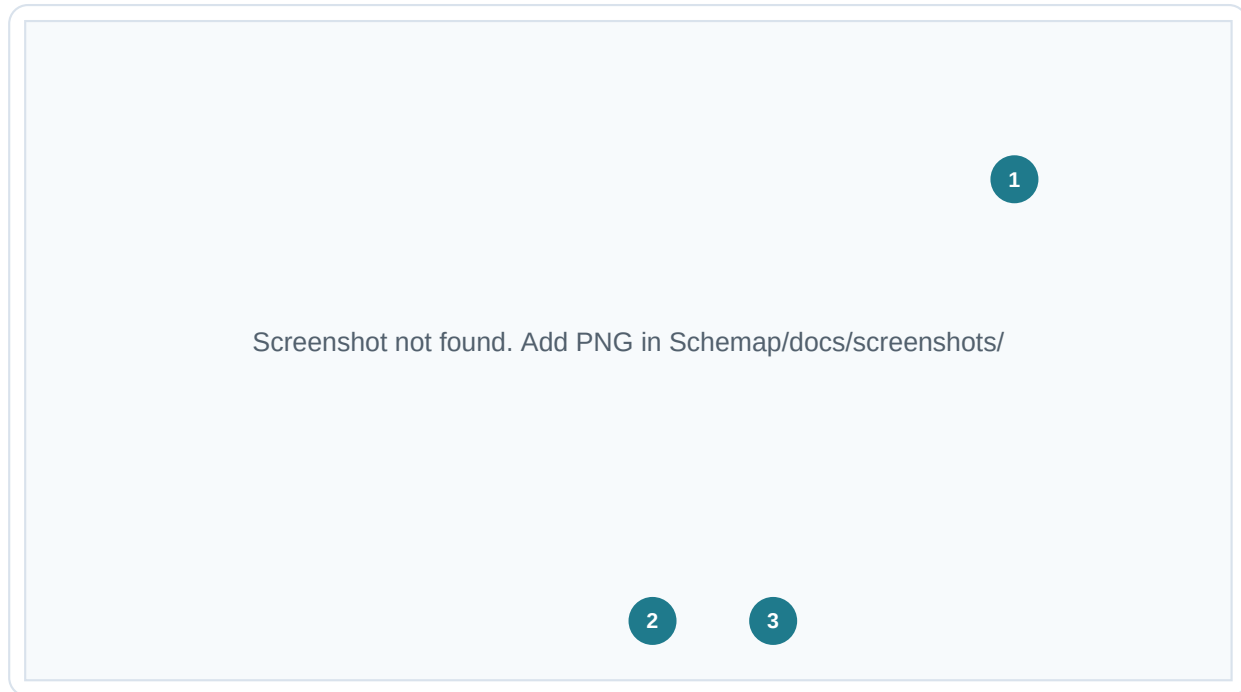


1	Selected block: Current block is focused for editing.
2	Inline inputs: Fill placeholders directly in the canvas.
3	Reorder flow: Drag blocks to change compiled order.

Expected screenshot file(s): 02-block-input-editing.png

Preview, Copy, And Export

Compiled output panel and quick export actions.

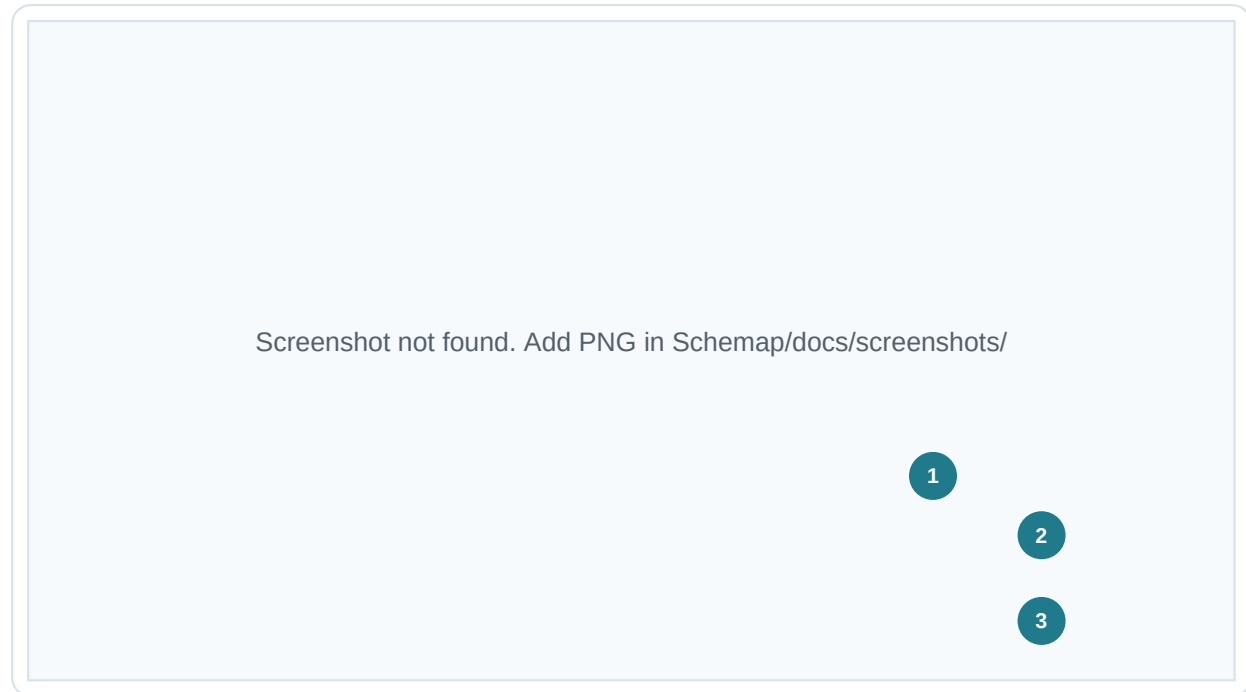


1	Compiled preview: Live output generated from current blocks.
2	Copy: Copy compiled prompt to clipboard.
3	Export: Export prompt as PDF/Markdown/Text.

Expected screenshot file(s): 03-preview-copy-export.png

Registry Workflow

Reusable object management for projects, snippets, variables, personas, and policies.



1	Object type: Switch registry category in the picker.
2	Object list: Select a reusable object.
3	Inspector/editor: View and edit object details.

Expected screenshot file(s): 04-registry-projects-snippets.png

Command Palette

Keyboard-first command discovery and execution.

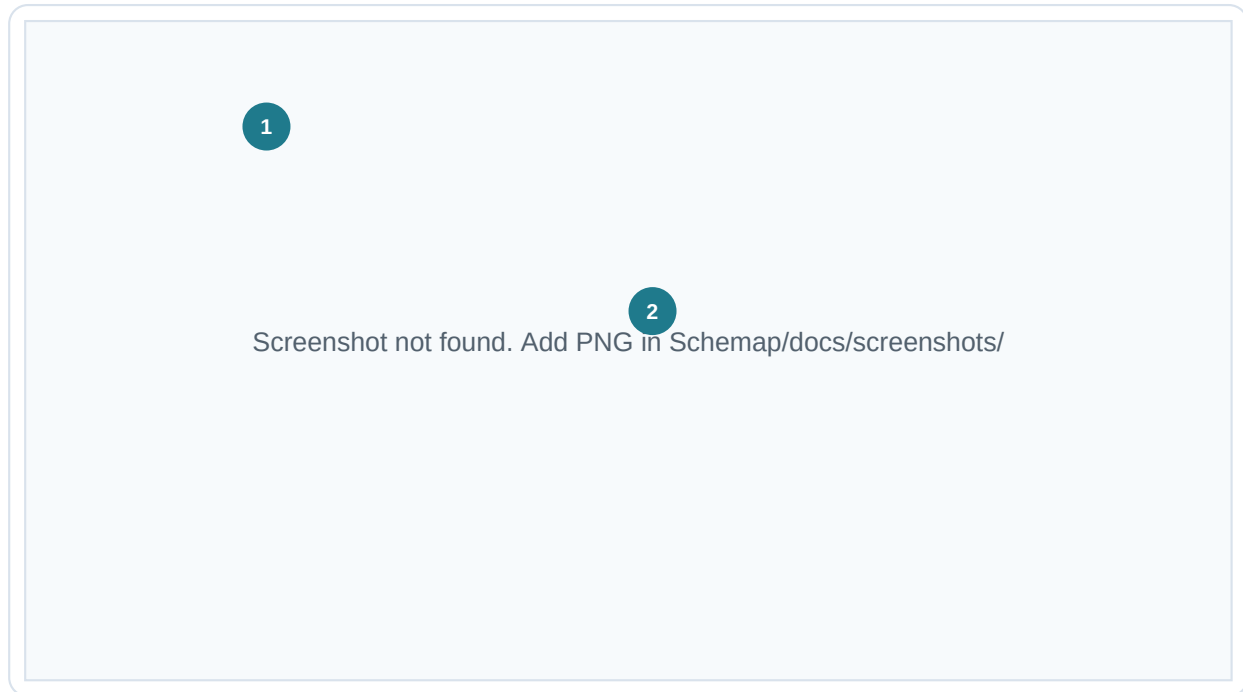


1	Search: Filter commands by name.
2	Command results: Run common actions without menu navigation.

Expected screenshot file(s): 05-command-palette.png

Variants Tab

Generate and compare alternative prompt versions.



1	Variant controls: Generate alternatives from current prompt.
2	Variant content: Inspect the selected variant output.

Expected screenshot file(s): 06-variants-tab.png

Exports Tab

Track and configure prompt exports.



1	Format options: Choose output format and export mode.
2	Export history: Review recent exports and destinations.

Expected screenshot file(s): 07-exports-tab.png

User Guide

This guide will help you learn how to use Schema to build, manage, and compose AI prompts.

Visual Walkthrough Placeholders

Use these placeholder slots until final product screenshots are available.

Placeholder 1: Build tab overview

- **Future image path:** Schemap/docs/screenshots/01-build-overview.png
- **Capture target:** Unified window on Build tab with all three panes visible.
- **What should be visible:** Block Library (left), Canvas (center), Preview + Registry (right).

Placeholder 2: Block add and inline input editing

- **Future image path:** Schemap/docs/screenshots/02-block-input-editing.png
- **Capture target:** Canvas with multiple blocks and active placeholder inputs.
- **What should be visible:** Drag/drop ordering, selected block state, inline input controls.

Placeholder 3: Preview and compile actions

- **Future image path:** Schemap/docs/screenshots/03-preview-copy-export.png
- **Capture target:** Right preview pane with compiled output and top-level actions.
- **What should be visible:** Compiled prompt text, token estimate, Copy / Export actions.

Placeholder 4: Registry workflow

- **Future image path:** Schemap/docs/screenshots/04-registry-projects-snippets.png
- **Capture target:** Registry section with one object type selected (for example Projects).
- **What should be visible:** Object list, selection, inspector/edit state for registry data.

Placeholder 5: Command palette and keyboard-first flow

- **Future image path:** Schemap/docs/screenshots/05-command-palette.png
- **Capture target:** Command palette opened with filtered commands.
- **What should be visible:** Search query, command categories, highlighted command and shortcut hints.

Placeholder 6: Variants tab

- **Future image path:** Schemap/docs/screenshots/06-variants-tab.png
- **Capture target:** Variants tab with generated variants and comparison context.
- **What should be visible:** Variant list/cards, selected variant details, compare action affordances.

Placeholder 7: Exports tab/history

- **Future image path:** Schemap/docs/screenshots/07-exports-tab.png
- **Capture target:** Exports tab showing format options or export history.
- **What should be visible:** Export format controls and recent export entries if present.

Application Overview

Schema is a visual prompt engineering studio that allows you to build prompts by arranging blocks on a canvas. Each block represents a component of your prompt (like objectives, roles, instructions, constraints, etc.), and you can fill in placeholders to customize the content.

Interface Overview

Schema uses a three-pane layout optimized for prompt composition:

Left Pane: Block Library

The block library contains all available blocks organized by category. Each category is color-coded for easy identification:

- **Intent & Framing** (Indigo) - Objectives, audience, use cases
- **Role & Perspective** (Brown) - Roles, personas, authority levels
- **Inputs & Context** (Blue) - Variables, files, external context
- **Instructions** (Purple) - Primary tasks, subtasks, priorities
- **Constraints & Guardrails** (Red) - Rules, formats, forbidden content
- **Tone & Style** (Cyan) - Tone, writing style, verbosity
- **Structure & Layout** (Orange) - Headings, sections, tables
- **Logic & Control** (Teal) - Conditionals, guards, fallbacks
- **Reasoning & Process** (Mint) - Analysis modes, decision frameworks
- **Verification & Quality** (Green) - Consistency checks, validators
- **Transforms** (Pink) - Rewrite, summarize, extract operations
- **Reuse & Governance** (Gray) - Snippets, policies, house styles
- **Execution & Control** (Yellow) - Execution notes, token budgets

Center Pane: Composition Canvas

The canvas is where you build your prompt by arranging blocks. Blocks can be:

- Dragged from the library
- Reordered by dragging
- Edited inline
- Deleted
- Selected to view properties

Right Pane: Preview and Assets

The right pane is split into two sections:

Top Section - Preview:

- Shows the compiled prompt output
- Token estimation (heuristic)
- Copy/export actions live in the canvas header

Bottom Section - Asset/Object Registry:

- Manage Projects, Snippets, Variables, Personas, and Policies
- Select objects to apply their context to your prompt

Getting Started Tutorial

Step 1: Add Your First Block

1. Look at the left sidebar (Block Library)
2. Find the "Intent & Framing" category (indigo color)
3. Click the "+" button next to "Objective" block, OR
4. Drag the "Objective" block to the center canvas

The block will appear on the canvas with placeholders ready to fill.

Step 2: Fill in Placeholders

1. Click on the "Objective" block on the canvas
2. You'll see input fields for:
 - goal - The main objective
 - priority - Priority level
 - must_have - Required elements
 - nice_to_have - Optional elements
3. Fill in these fields with your content
4. The preview on the right will update automatically

Step 3: Add More Blocks

1. Add a "Role" block from "Role & Perspective" category
2. Fill in the role placeholder (e.g., "Senior Software Engineer")
3. Add a "Primary Instruction" block from "Instructions" category
4. Fill in the task placeholder

Step 4: View the Preview

1. Look at the right pane preview
2. The compiled prompt updates as you edit blocks and inputs
3. Watch the token estimate for quick size feedback

Step 5: Reorder Blocks

1. Drag blocks up or down on the canvas to reorder them
2. The prompt order follows the block order on the canvas
3. The preview updates automatically

Block Library Usage

Adding Blocks

Method 1: Plus Button

- Click the "+" button next to any block in the library
- The block is added to the end of the canvas

Method 2: Drag and Drop

- Drag a block from the library to the canvas
- Drop it at the desired position (you'll see an insertion indicator)
- The block is inserted at that position

Block Compatibility

Some blocks work better together than others. The library shows compatibility indicators:

- **High** (Green) - Blocks work very well together
- **Med** (Orange) - Blocks work reasonably well together
- **Low** (Gray) - Blocks have limited compatibility

When you select a block on the canvas, compatible blocks are highlighted in the library.

Block Types

Each block has a type that determines how it's used:

- **String** - Single-line text input
- **Boolean** - True/false conditions
- **Object** - Multi-line or structured content

Canvas Composition Workflow

Building a Prompt

1. **Start with Intent:** Add Objective, Audience, or Use Case blocks
2. **Define Role:** Add Role or Persona blocks
3. **Specify Instructions:** Add Primary Instruction and Subtask blocks
4. **Add Constraints:** Add Format Constraint, Length Constraint, or Forbidden Content blocks
5. **Set Tone:** Add Tone or Writing Style blocks
6. **Structure Output:** Add Heading, Section, or Table Schema blocks
7. **Add Logic:** Add If/Else or Guard blocks if needed
8. **Include Verification:** Add Consistency Check or Output Scorer blocks

Editing Blocks

1. Click a block on the canvas to select it
2. Fill in or edit placeholders in the input fields
3. For object-type blocks, use the multi-line text editor

4. Changes are reflected immediately in the preview

Reordering Blocks

- Drag blocks vertically to reorder
- A blue insertion line shows where the block will be placed
- The prompt order follows the visual order on the canvas

Deleting Blocks

- Select a block and press Delete key, OR
- Drag a block from the canvas back to the library sidebar
- The block and its inputs are removed

Block Selection

- Click a block to select it (highlighted border)
- Click again to deselect
- Click empty canvas area to deselect all

Clipboard and Editing

- **Copy Block (Cmd+Shift+C)**: Copies the selected block and its inputs to the clipboard
- **Cut Block (Cmd+X)**: Copies the selected block and removes it from the canvas
- **Paste Block (Cmd+V)**: Pastes the last copied/cut block at the end of the canvas and selects it
- **Duplicate Block (Cmd+D)**: Inserts a copy of the selected block right after it and selects the duplicate

Preview Stage

The preview pane shows the compiled prompt output and updates as you edit blocks and inputs. Token estimation is a heuristic (~4 characters per token) to help you gauge size.

Note: Diff/Filled/Raw preview modes are not currently exposed in the UI.

Asset/Object Registry

The bottom right pane manages reusable prompt components.

Projects

Define project-level settings:

- Objectives and domain context
- Default audience profiles
- Global tone and style
- Quality bars and risk sensitivity
- Token budgets

Snippets

Reusable instruction blocks that can be referenced in prompts:

- Instruction snippets

- Constraint snippets
- Structure templates
- Reasoning patterns
- Examples and negative examples

Variables

Manage prompt variables:

- User inputs
- Constants
- Computed values
- Context flags
- Environment variables

Personas

Define role personas:

- Role definitions
- Expertise maps
- Decision styles (Analytical, Heuristic, Conservative)
- Communication styles
- Ethical boundaries

Policies

Governance rules:

- Content policies
- Compliance rules
- Output requirements
- Reasoning constraints
- Escalation paths

Using Objects in Prompts

1. Select an object type from the picker (Projects, Snippets, etc.)
2. Click on an object in the list
3. The object's context is appended to your compiled prompt
4. The preview updates to show the combined result

Exporting Prompts

Copy to Clipboard

1. Click the "Copy" button in the canvas toolbar
2. The compiled template is copied to your clipboard
3. Paste it wherever you need it

Export from the Canvas Toolbar

1. Click the "Export" button in the canvas toolbar
2. Choose an export format:
 - **PDF** (text-only export of the compiled prompt)
 - **Markdown (.md)**
 - **Plain Text (.txt)**
3. Select a save location
4. The compiled prompt is saved to a file

Export from the File Menu

1. Choose **File** → **Export...**
2. Select a format:
 - **Plain Text (.txt)**
 - **Markdown (.md)**
 - **JSON (.json)** (includes title/notes, block metadata, and block inputs)
3. Select a save location
4. The compiled prompt or JSON snapshot is saved to a file

Best Practices

Prompt Structure

1. **Start with Context:** Begin with Objective, Audience, or Use Case
2. **Define Role Early:** Set the role or persona before instructions
3. **Be Specific:** Use concrete placeholders rather than vague terms
4. **Add Constraints:** Explicitly state what should NOT be included
5. **Structure Output:** Use Structure blocks to define expected format
6. **Verify Quality:** Add verification blocks for important prompts

Block Organization

1. Group related blocks together
2. Place constraints near the instructions they constrain
3. Put verification blocks at the end
4. Use execution blocks for meta-instructions

Placeholder Naming

- Use descriptive names: goal not g
- Use snake_case: expertise_level not expertiseLevel

- Be consistent across blocks

Token Management

- Monitor token estimates in the preview
- Use Token Budget Guard blocks to set limits
- Break large prompts into smaller, focused prompts when possible

Reusability

- Create Snippets for commonly used instruction patterns
- Define Variables for values used across multiple prompts
- Use Projects to set defaults for related prompts
- Create Personas for consistent role definitions

Keyboard Shortcuts

Schema includes keyboard-first navigation, focus management, a command palette, and a searchable keyboard shortcuts window.

Command Palette

- Cmd+/- - Open the command palette
- Type to filter commands; Up/Down to move; Enter to run; Esc to close
- Commands include New, Open, Save, Export, Add Block, Compile, and Focus actions

Pane Focus

- Tab / Shift+Tab - Cycle focus between Library → Canvas → Preview → Registry
- Cmd+Option+L - Focus Block Library
- Cmd+Option+C - Focus Canvas
- Cmd+Option+V - Focus Preview
- Cmd+Option+R - Focus Registry

Block Library

- j/k or Up/Down - Move focus
- Enter - Add focused block to the canvas
- / - Search blocks
- Esc - Exit search

Canvas

- j/k or Up/Down - Move block selection
- gg - First block
- G - Last block
- Enter - Edit selected block
- Esc - Clear selection

Registry

- j/k or Up/Down - Move selection

- Enter - Open inspector for selected object
- n - Create new object (current registry type)
- d - Delete selected object
- / - Search registry
- Esc - Exit search or clear selection

Advanced Features

Compatibility System

The compatibility system suggests blocks that work well together:

1. Select a block on the canvas
2. Compatible blocks in the library are highlighted
3. Click "Add" on a compatible block to insert it after the selected block
4. Compatibility levels (High/Med/Low) are shown as badges

Shadow Blocks

When a block is selected, compatible blocks appear as "shadow" suggestions. Click to add them directly after the selected block.

Block Inputs

- String blocks use single-line text fields
- Object blocks use multi-line text editors
- Boolean blocks use conditional logic syntax
- Some blocks auto-detect input type based on content

Troubleshooting

Preview Not Updating

- Check that placeholders are filled correctly
- Verify blocks are in the correct order
- Give the preview a moment to update after edits (debounced compilation)
- Check [\[Troubleshooting Guide\]\(TROUBLESHOOTING.md\)](#) for more

Blocks Not Appearing

- Verify the block library is loaded (check left sidebar)
- Try restarting the application
- Check [\[Known Issues\]\(KNOWN_ISSUES.md\)](#)

Placeholders Not Replacing

- Ensure placeholder names match exactly (case-sensitive)
- Check for typos in placeholder names
- Verify the placeholder syntax: `{{name}}` not `{name}` or `[[name]]`

Next Steps

- Read the [Block Reference](BLOCK_REFERENCE.md) for detailed information about all blocks
- Check [Examples](EXAMPLES.md) for real-world prompt compositions
- Review [Architecture](ARCHITECTURE.md) to understand how the system works

Troubleshooting Guide

Common issues, error messages, and solutions for Schema.

Common Issues

Application Won't Launch

Symptoms: Application crashes immediately on launch

Possible Causes:

1. SwiftData ModelContainer creation failed
2. Missing dependencies
3. Corrupted application state

Solutions:

1. Check Console.app for error messages

- Look for "Could not create ModelContainer" error
- Check for SwiftData-related errors

2. Verify System Requirements

- macOS 13.0 or later
- Xcode 15.0 or later (for development)

3. Clean Build

- In Xcode: Product > Clean Build Folder (Cmd+Shift+K)
- Rebuild and run

4. Check Code Signing

- Verify code signing is configured
- Check for certificate issues

If Issue Persists:

- Check [Known Issues](KNOWN_ISSUES.md) for SwiftData initialization and persistence limitations
- Report issue with Console.app logs

Blocks Don't Appear in Library

Symptoms: Left sidebar is empty or missing blocks

Possible Causes:

1. Block library not seeded
2. Application initialization failed
3. UI rendering issue

Solutions:**1. Restart Application**

- Quit and relaunch
- Check if blocks appear

2. Check Console Logs

- Look for errors in `seedLibrary()` method
- Check for initialization errors

3. Verify Block Library

- Blocks are seeded in `PromptStudioModel.init()`
- Check that `seedLibrary()` is called

If Issue Persists:

- Check [Known Issues](KNOWN_ISSUES.md)
- Verify block definitions are valid

Preview Not Updating

Symptoms: Preview pane doesn't update when blocks or inputs change

Possible Causes:

1. Compilation not triggered
2. `onChange` handlers not connected
3. State not updating

Solutions:**1. Check Block Inputs**

- Ensure placeholders are filled
- Verify input values are saved

2. Trigger Manual Compilation

- Add or remove a block
- This should trigger the debounced compilation path

3. Allow for Debounce

- Input updates are debounced; give the preview a moment after edits

4. Check Console Logs

- Look for compilation errors
- Check for state update issues

If Issue Persists:

- Check [Known Issues](KNOWN_ISSUES.md) for compilation issues
- Verify onChange handlers in CompositionCanvasView

Placeholders Not Replacing

Symptoms: Placeholders like `{{name}}` appear in compiled output instead of values

Possible Causes:

1. Placeholder name mismatch (case-sensitive)
2. Input not saved
3. Compilation error

Solutions:**1. Check Placeholder Names**

- Ensure exact match (case-sensitive)
- Example: `{{user_name}}` not `{{userName}}` or `{{User_Name}}`

2. Verify Input Values

- Check that inputs are filled in
- Verify values are saved in `blockInputs` dictionary

3. Check Placeholder Syntax

- Must be `{{name}}` not `{name}` or `[[name]]`
- No spaces inside braces (unless in name)

4. Test with Simple Block

- Use a simple block with one placeholder
- Verify replacement works

Example:

...

Snippet: `ROLE: {{role}}`

Input key: `role` (correct)

Input key: `Role` (wrong - case mismatch)

Input key: `user_role` (wrong - name mismatch)

...

If Issue Persists:

- Check [Known Issues](KNOWN_ISSUES.md) for placeholder replacement issues

- Verify placeholder extraction regex is working

Drag and Drop Not Working

Symptoms: Can't drag blocks or reorder blocks

Possible Causes:

1. Drag gesture not recognized
2. Drop destination not configured
3. BlockDragItem conversion failing

Solutions:

1. Check Drag Source

- Verify blocks are draggable in library
- Check for drag indicators

2. Check Drop Destination

- Verify canvas accepts drops
- Check for drop indicators

3. Try Different Method

- Use "+" button instead of drag
- Try clicking and dragging more slowly

4. Restart Application

- Sometimes drag state gets stuck
- Restart to reset

If Issue Persists:

- Check [Known Issues](KNOWN_ISSUES.md)
- Verify BlockDragItem implementation

Export Fails

Symptoms: Export button doesn't work or file not created

Possible Causes:

1. File permissions
2. Invalid file path
3. Export format error

Solutions:

1. Check File Permissions

- Verify write permissions for save location

- Try different save location

2. Use Copy Instead

- Use "Copy" button to copy to clipboard
- Paste into text editor and save manually

3. Check Export Format

- Try different export format
- Verify format is supported

4. Check Console Logs

- Look for file write errors
- Check for permission errors

If Issue Persists:

- Check [Known Issues](KNOWN_ISSUES.md) for export issues
- Verify export implementation

Performance Issues

Symptoms: Application is slow, laggy, or unresponsive

Possible Causes:

1. Too many blocks on canvas
2. Rapid input changes triggering many compilations
3. Large block library

Solutions:

1. Reduce Canvas Size

- Remove unnecessary blocks
- Keep canvas under 20 blocks if possible

2. Slow Down Input

- Type more slowly to reduce compilation frequency
- Wait for preview to update before continuing

3. Restart Application

- Close and reopen to clear memory
- May improve performance

4. Check System Resources

- Check Activity Monitor for memory usage
- Close other applications

If Issue Persists:

- Check [Known Issues](KNOWN_ISSUES.md) for performance issues

- Consider breaking large prompts into smaller ones

Error Messages

"Could not create ModelContainer"

Location: Schema/SchemaApp.swift:22

Meaning: SwiftData ModelContainer initialization failed

Causes:

- SwiftData not available on macOS version
- Schema definition error
- Data store corruption

Solutions:

1. Check macOS Version

- Requires macOS 13.0 or later
- Update macOS if needed

2. Clean Data Store

- Delete application data
- Restart application

3. Check Schema Definition

- Verify Item model is valid
- Check for schema errors

Note: This is a fatal error with no recovery. See [Known Issues](KNOWN_ISSUES.md).

Compilation Errors (Silent)

Location: Schema/ViewModels/PromptStudioModel.swift

Meaning: Compilation may fail silently

Symptoms:

- Partial templates
- Missing placeholders
- Malformed output

Solutions:

1. Check Block Snippets

- Verify snippets are valid
- Check for syntax errors

2. Check Placeholder Names

- Ensure placeholder names match

- Verify no typos

3. Review Compiled Output

- Check preview for issues
- Manually fix if needed

Debugging Tips

Enable Debug Logging

Add print statements to track state:

```
```swift

func compileNow() {
 print("Compiling with \(canvasBlocks.count) blocks")
 // ... compilation code
 print("Compiled template: \(compiledTemplate.prefix(100))")
}

```
```

Check State in Debugger

1. Set breakpoints in key methods:

- `compileNow()` / `compileDebounce()`
- `addToCanvas()`
- `compatibleItems(for:)`

2. Inspect state:

- `canvasBlocks` array
- `blockInputs` dictionary
- `compiledTemplate` string

Use SwiftUI Previews

Test individual components:

```
```swift

#Preview {

 BlockLibraryView(model: PromptStudioModel(), onToggleSidebar: {})

}

```
```

Check Console.app

1. Open Console.app
2. Filter for "Schema" or process name
3. Look for errors or warnings
4. Check for SwiftData errors

Recovery Procedures

Data Loss Prevention

Before Troubleshooting:

1. Export current prompt
2. Copy compiled template to clipboard
3. Save to external file

If Application Crashes:

1. Check if auto-save exists (currently none)
2. Check for crash logs
3. Restore from exported files if available

Resetting Application State

If State is Corrupted:

1. Quit application
2. Delete application data (if SwiftData persists)
3. Restart application
4. Rebuild prompt from exported files

Note: Currently no persistence, so restart clears all state.

Recovering from Errors

Compilation Errors:

1. Check preview output
2. Identify problematic blocks
3. Remove or fix blocks
4. Recompile

UI Errors:

1. Restart application
2. Rebuild canvas

3. Check for UI state issues

Getting Help

Information to Provide

When reporting issues, include:

1. **Error Messages:** Full error text
2. **Steps to Reproduce:** Detailed steps
3. **Expected Behavior:** What should happen
4. **Actual Behavior:** What actually happens
5. **Environment:** macOS version, Xcode version
6. **Console Logs:** Relevant log entries
7. **Screenshots:** If applicable

Resources

- [Known Issues](KNOWN_ISSUES.md) - Known problems and limitations
- [User Guide](USER_GUIDE.md) - How to use the application
- [Architecture](ARCHITECTURE.md) - System design
- [Developer Guide](DEVELOPER_GUIDE.md) - For developers

Prevention

Best Practices

1. **Export Regularly:** Save work frequently
2. **Keep Canvas Small:** Avoid too many blocks
3. **Validate Inputs:** Check placeholder names match
4. **Test Incrementally:** Test as you build
5. **Backup Work:** Keep copies of important prompts

Avoiding Common Issues

1. **Placeholder Names:** Use exact matches, snake_case
2. **Block Order:** Verify order is correct
3. **Input Values:** Fill all required placeholders
4. **Performance:** Keep canvas size reasonable
5. **Export:** Export before closing

See Also

- [Known Issues](KNOWN_ISSUES.md) - Detailed issue tracking
- [User Guide](USER_GUIDE.md) - Usage instructions
- [Architecture](ARCHITECTURE.md) - System understanding